

Project Report

Nikoleta Xenaki, csd4968

Introduction

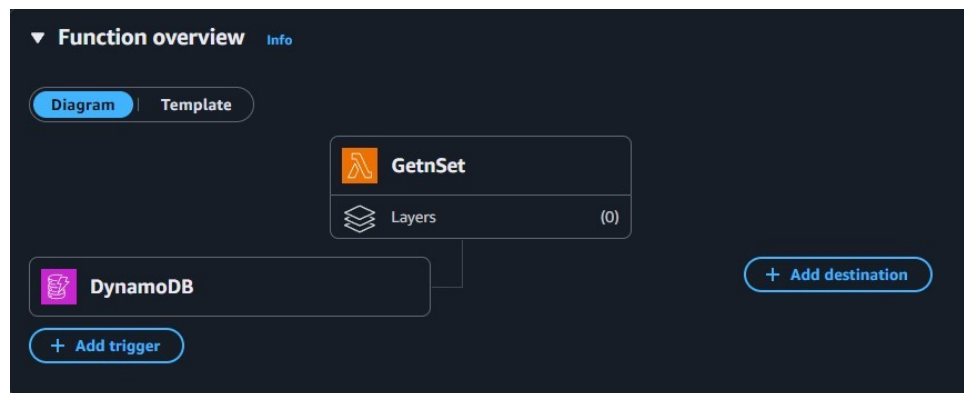
In this project we calculate how three users allocate cloud resources (virtual machines/processors) in order to maximize their utility function (minimize their cost and the execution time of their tasks).

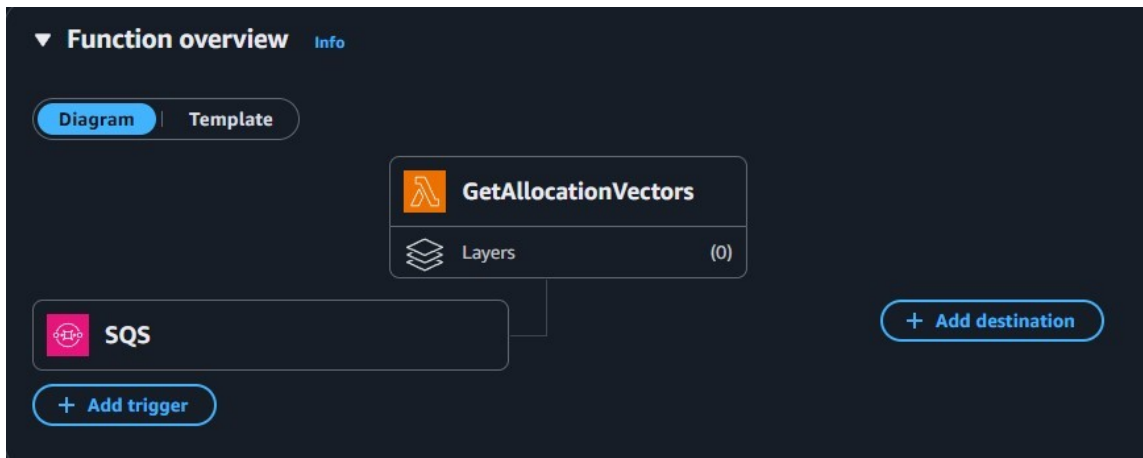
Each user calculates their allocation matrix of reserved resources (initially ignoring the other participants) and sends it to the resource manager. Then the manager combines their matrices, calculates the real cost and time for each user according to each resource and sends the resulting T and E matrices back to the users. Afterwards each user calculates their actual utility function minding the other users too and the new time matrix ($t_{\text{hat_new}}$) in order to begin round two, which follows as described above. Note that **if a user's utility for each possible allocation matrix "b" is 0** then the selecting matrix is [0,0,0,0,0]. *For the overall implementation we used python Flask and AWS services for cloud computation.*

Cloud Implementation

For the implementation of such execution in the AWS cloud we used:

- (1) An API Gateway to match the request paths to the correct EC2 IP endpoints where the scripts are running (<https://t3u455mheh.execute-api.us-east-1.amazonaws.com>).
- (2) Three virtual machine instances (EC2s) simulating users, running users' python flask script waiting for requests.
- (3) One virtual machine (EC2) simulating the resource manager who, as input receives users' allocation strategies and proceeds it's execution as described earlier.
- (4) A DynamoDB database to store the user's allocation matrices according to the round.
- (5) A Simple Queue data structure where users forward their allocation matrices.
- (6) A lamda function that is triggered by each message submitted in the above described queue, and stores users' information and allocation matrices to the DynamoDB instance.
- (7) A lamda function that is triggered by the DynamoDB streams and when all three matrices, at each round, are submitted by the users it combines them and sends the resulting matrix to the resource manager.





Firstly we ssh to the EC2s and run the users' and resource manager's flask script inside a python virtual environment.

```

asus_nikoleta_deb@NikoletaXenaki-ASUS: ~
asus_nikoleta_deb@NikoletaXenaki-ASUS:~$ ssh -i ~/.ssh/rest.pem ubuntu@100.49.5.69
asus_nikoleta_deb@NikoletaXenaki-ASUS:~$ ssh -i ~/.ssh/rest.pem ubuntu@100.49.242.96
asus_nikoleta_deb@NikoletaXenaki-ASUS:~$ ssh -i ~/.ssh/rest.pem ubuntu@18.211.101.85

```

User_1

User_2

User_3

Resource Manager

<pre> ubuntu@ip-172-31-26-43:~/user1\$ source .venv/bin/activate (.venv) ubuntu@ip-172-31-26-43:~/user1\$ python3 user1.py * Serving Flask app 'user1' * Debug mode: off WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead. * Running on http://127.0.0.1:5000 * Running on http://172.31.26.43:5000 Press CTRL+C to quit ^[[A^[[A </pre>	<pre> ubuntu@ip-172-31-25-131:~/user2\$ source .venv/bin/activate (.venv) ubuntu@ip-172-31-25-131:~/user2\$ python3 user2.py * Serving Flask app 'user2' * Debug mode: off WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead. * Running on http://127.0.0.1:5000 * Running on http://172.31.25.131:5000 Press CTRL+C to quit </pre>	<pre> ubuntu@ip-172-31-22-139:~/user3\$ source .venv/bin/activate (.venv) ubuntu@ip-172-31-22-139:~/user3\$ python3 user3.py * Serving Flask app 'user3' * Debug mode: off WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead. * Running on http://127.0.0.1:5000 * Running on http://172.31.22.139:5000 Press CTRL+C to quit </pre>	<pre> (.venv) ubuntu@ip-172-31-31-184:~/manager\$ python3 resource_manager.py * Serving Flask app 'resource_manager' * Debug mode: on WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead. * Running on http://127.0.0.1:5000 Press CTRL+C to quit * Restarting with stat * Debugger is active! * Debugger PIN: 521-141-205 </pre>
---	---	---	--

Secondly we use Postman Agent to send API requests to each user separately, in order for them to start their execution and calculate their allocation matrix, which they will eventually submit to the queue.

POST https://t3u455mheh.e: + No environment

https://t3u455mheh.execute-api.us-east-1.amazonaws.com/user2 Save Share

POST https://t3u455mheh.execute-api.us-east-1.amazonaws.com/user2 Send

Docs Params Authorization Headers (8) Body Scripts Tests Settings Cookies

Query Params

	Key	Value	Description	Bulk Edit
	Key	Value	Description	

Body Cookies Headers (6) Test Results 200 OK 416 ms 226 B

{ } JSON Preview Visualize

```
1 {
2   "status": "started"
3 }
```

POST https://t3u455mheh.e: + No environment

https://t3u455mheh.execute-api.us-east-1.amazonaws.com/user3 Save Share

POST https://t3u455mheh.execute-api.us-east-1.amazonaws.com/user3 Send

Docs Params Authorization Headers (8) Body Scripts Tests Settings Cookies

Query Params

	Key	Value	Description	Bulk Edit
	Key	Value	Description	

Body Cookies Headers (6) Test Results 200 OK 625 ms 226 B

{ } JSON Preview Visualize

```
1 {
2   "status": "started"
3 }
```

For now with users' 2 and 3 actual ids being 1 and 2, the users' table with their allocated strategies looks like this:

Table: User_Strategies - Items returned (3)

Actions Create item

Scan started on December 29, 2025, 00:52:07

	user_id (String)	round_id (Number)	done	matrix
☐	2	1		[{"N": "1" ...
☐	1	1		[{"N": "0" ...
☐	AGGREGATE	1	false	

Notice that this extra row (AGGREGATE) is inserted, each round, by the first user submitting their strategy and acts as a lock, that signals whether each round's data are sent successfully to the resource manager. Thus the first user to submit their strategy is the one that waits for the others to submit theirs too in order to continue the execution. The done column in AGGREGATE row is written "true" only when all three allocation matrices have been sent to the resource manager.

Continuing...

The screenshot shows a REST client interface with a POST request to `https://t3u455mheh.execute-api.us-east-1.amazonaws.com/user1`. The response is a 200 OK status with a response time of 365 ms and a body size of 226 B. The response body is a JSON object: `{ "status": "started" }`.

Now after the last user's request, all the information (**for both rounds**) about the allocation strategies selected by users, the computation of initial and actual utility value of users, along with the computed matrices (T , E and $t_{\text{hat_new}}$) will be depicted as:

```

Running on http://172.31.26.43:5000
* Running on http://172.31.26.43:5000
Press CTRL+C to quit
----- Initial utility: 0.16 -----
Matrix: [0, 0, 0, 1, 1]
3.235.32.192 - - [09/Jun/2026 14:16:39] "POST /start HTTP/1.1" 200 -
----- Actual utility value: 0.0 -----
NEW t_hat: [[7.33, 10.13, 8.53, 12.8, 4.0], [6.33, 9.33, 8.13, 12.3, 3.8], [5.33, 8.63, 7.73, 12.1, 3.4]]
Matrix: [1, 0, 0, 0, 1]
3.233.96.196 - - [09/Jun/2026 14:16:57] "POST /result HTTP/1.1" 200 -
----- Actual utility value: 0.0 -----
3.233.96.196 - - [09/Jun/2026 14:16:57] "POST /result HTTP/1.1" 200 -

Running on http://172.31.23.151:5000
Press CTRL+C to quit
----- Initial utility: 0.13 -----
Matrix: [0, 1, 1, 1, 0]
3.235.36.252 - - [09/Jun/2026 14:16:00] "POST /start HTTP/1.1" 200 -
----- Actual utility value: 0.09 -----
NEW t_hat: [[7.33, 10.13, 8.53, 12.8, 4.0], [6.33, 9.33, 8.13, 12.3, 3.8], [5.33, 8.63, 7.73, 12.1, 3.4]]
Matrix: [1, 1, 0, 0, 1]
3.233.96.196 - - [09/Jun/2026 14:16:57] "POST /result HTTP/1.1" 200 -
----- Actual utility value: 0.06 -----
3.233.96.196 - - [09/Jun/2026 14:16:57] "POST /result HTTP/1.1" 200 -

Running on http://172.31.22.139:5000
Press CTRL+C to quit
----- Initial utility: 0.12 -----
Matrix: [1, 1, 1, 1, 0]
3.235.33.176 - - [09/Jun/2026 14:16:55] "POST /start HTTP/1.1" 200 -
----- Actual utility value: 0.09 -----
NEW t_hat: [[7.33, 10.13, 8.53, 12.8, 4.0], [6.33, 9.33, 8.13, 12.3, 3.8], [5.33, 8.63, 7.73, 12.1, 3.4]]
No valid matrices found
Matrix: [0, 0, 0, 0, 0]
3.233.96.196 - - [09/Jun/2026 14:16:57] "POST /result HTTP/1.1" 200 -
----- Actual utility value: 0.0 -----
3.233.96.196 - - [09/Jun/2026 14:16:57] "POST /result HTTP/1.1" 200 -

T: [[0, 0, 0, 10.5, 3.0], [0, 8.4, 7.2, 9.0, 0], [4.0, 7.0, 6.4, 8.4, 0]]
E: [[0, 0, 0, 6.3, 6.0], [0, 5.04, 5.4, 5.4, 0], [4.0, 4.2, 4.8, 5.04, 0]]
Sent results to user 0, status 200
Sent results to user 1, status 200
Sent results to user 2, status 200
107.21.67.215 - - [09/Jun/2026 14:16:57] "POST /compute HTTP/1.1" 200 -
T: [[14.66, 0, 0, 8.0], [12.66, 9.33, 0, 0, 7.6], [0, 0, 0, 0, 0]]
E: [[7.33, 0, 0, 0, 8.0], [6.33, 11.2, 0, 0, 7.6], [0, 0, 0, 0, 0]]
Sent results to user 0, status 200
Sent results to user 1, status 200
Sent results to user 2, status 200
107.21.67.215 - - [09/Jun/2026 14:16:57] "POST /compute HTTP/1.1" 200 -

```

As we can see the same pattern of results is depicted for both rounds, and the last snapshot of our DynamoDB instance looks like:

Table: User_Strategies - Items returned (8)

Scan started on December 29, 2025, 01:07:53

< 1 > ⚙

☐	user_id (String) ▾	round_id (Number) ▾	done ▾	matrix ▾
☐	<u>2</u>	1		[{"N": "1" ...
☐	<u>2</u>	2		[{"N": "1" ...
☐	<u>1</u>	1		[{"N": "0" ...
☐	<u>1</u>	2		[{"N": "1" ...
☐	<u>0</u>	1		[{"N": "0" ...
☐	<u>0</u>	2		[{"N": "1" ...
☐	<u>AGGREGATE</u>	1	true	
☐	<u>AGGREGATE</u>	2	true	

All in all the matrices through the execution:

Step 1

User (Task)	Allocation Vector	Expected Utility	Actual Utility
S ₁	(0, 0, 0, 1, 1)	0.16	0.09
S ₂	(0, 1, 1, 1, 0)	0.13	0.09
S ₃	(1, 1, 1, 1, 0)	0.12	0.09

T				
R1	R2	R3	R4	R5
0	0	0	10.5	3
0	8.4	7.2	9	0
4	7	6.4	8.4	0

E

R1	R2	R3	R4	R5
0	0	0	6.3	6
0	5.04	5.4	5.4	0
4	4.2	4.8	5.04	0

new_That

R1	R2	R3	R4	R5
7.33	10.13	8.53	12.8	4
6.33	9.33	8.13	12.3	3.8
5.33	8.63	7.73	12.1	3.4

Step 2

User (Task)	Allocation Vector	Expected Utility	Actual Utility
S ₁	(1, 0, 0, 0, 1)	0.10	0.07
S ₂	(1, 1, 0, 0, 1)	0.07	0.06
S ₃	(0, 0, 0, 0, 0)	0	0

T

R1	R2	R3	R4	R5
14.66	0	0	0	8
12.66	9.33	0	0	7.6
0	0	0	0	0

E

R1	R2	R3	R4	R5
7.33	0	0	0	8
6.33	11.2	0	0	7.6
0	0	0	0	0

Thank you for your time!
The end